Airbnb Price Prediction — Complete ML Pipeline

AIGC 5003 — Machine Learning in Cloud Computing

This notebook implements a complete machine learning pipeline to predict Airbnb nightly listing prices using Amazon SageMaker.

Student: Haneen altaie **Dataset:** Airbnb listings — 953 rows
Target variable: price(in_dollar)
Region: Canada Central (ca-central-1)

Pipeline Overview:

Step	Description
1	Data Loading & Analysis
2	Data Cleaning & Feature Engineering (537 features including season & quarter)
3	BM1 — Baseline XGBoost default HP → validation:rmse = 71.84
4	BM2 — XGBoost after Bayesian HP Tuning → validation:rmse = 67.65
5	BM3 — Random Forest Deployment (ml.m5.large)
6	Inferencing & Evaluation → RMSE = 71.62, MAE = 52.64, R^2 = 0.2655
7	Resource Cleanup
8	Reflection & Data Insights

In [2]:
```
# Import all required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import re
import tarfile
import joblib
import boto3
import sagemaker

from sagemaker import get_execution_role
from sagemaker.inputs import TrainingInput
from sagemaker.image_uris import retrieve
from sagemaker.tuner import IntegerParameter, ContinuousParameter, HyperparameterTu
```

```
from sagemaker.sklearn.model import SKLearnModel
from sagemaker.serializers import CSVSerializer
from sagemaker.deserializers import StringDeserializer

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
from sklearn.preprocessing import LabelEncoder

# SageMaker session setup
session = sagemaker.Session()
bucket  = session.default_bucket()
role    = get_execution_role()
region  = session.boto_region_name

print("✅ Libraries loaded successfully")
print(f"Bucket: {bucket}")
print(f"Role:   {role}")
print(f"Region: {region}")
```

```
sagemaker.config INFO - Not applying SDK defaults from location: /etc/xdg/sagemake
r/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location: /home/ec2-user/.c
onfig/sagemaker/config.yaml
✅ Libraries loaded successfully
Bucket: sagemaker-ca-central-1-263245924849
Role:   arn:aws:iam::263245924849:role/service-role/AmazonSageMaker-ExecutionRole-
20260323T193815
Region: ca-central-1
```

Step 1 — Data Loading & Analysis

We load two datasets:

File	Description
`final_airbnb_ml.csv`	Preprocessed file — 953 rows, 671 columns with anonymous boolean features
`airbnb.csv`	Original raw dataset — 953 rows, 7 meaningful columns

Both files are stored locally in the SageMaker JupyterLab environment.

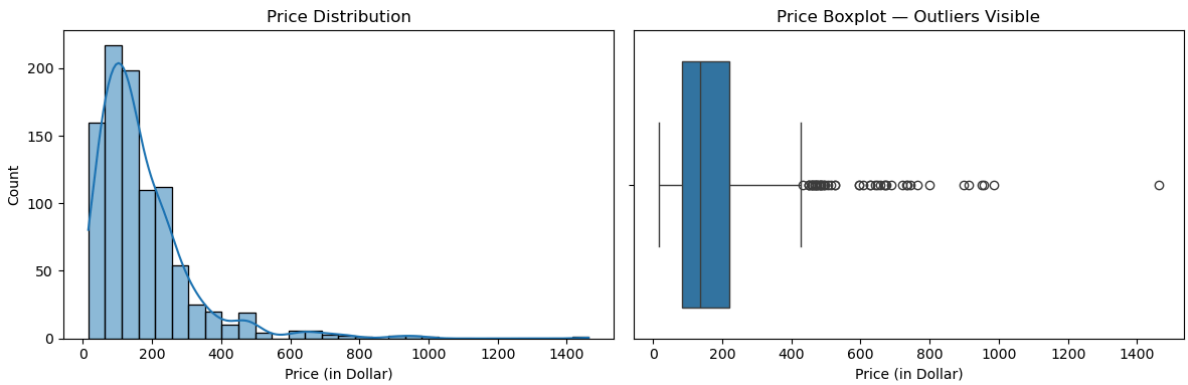
The preprocessed file is used for the SageMaker built-in training job.
The raw file is used for feature engineering to build a better model. The preprocessed file is used for Benchmark 1 (BM1) baseline training. The raw file is used for feature engineering to build a better model (BM2, BM3).

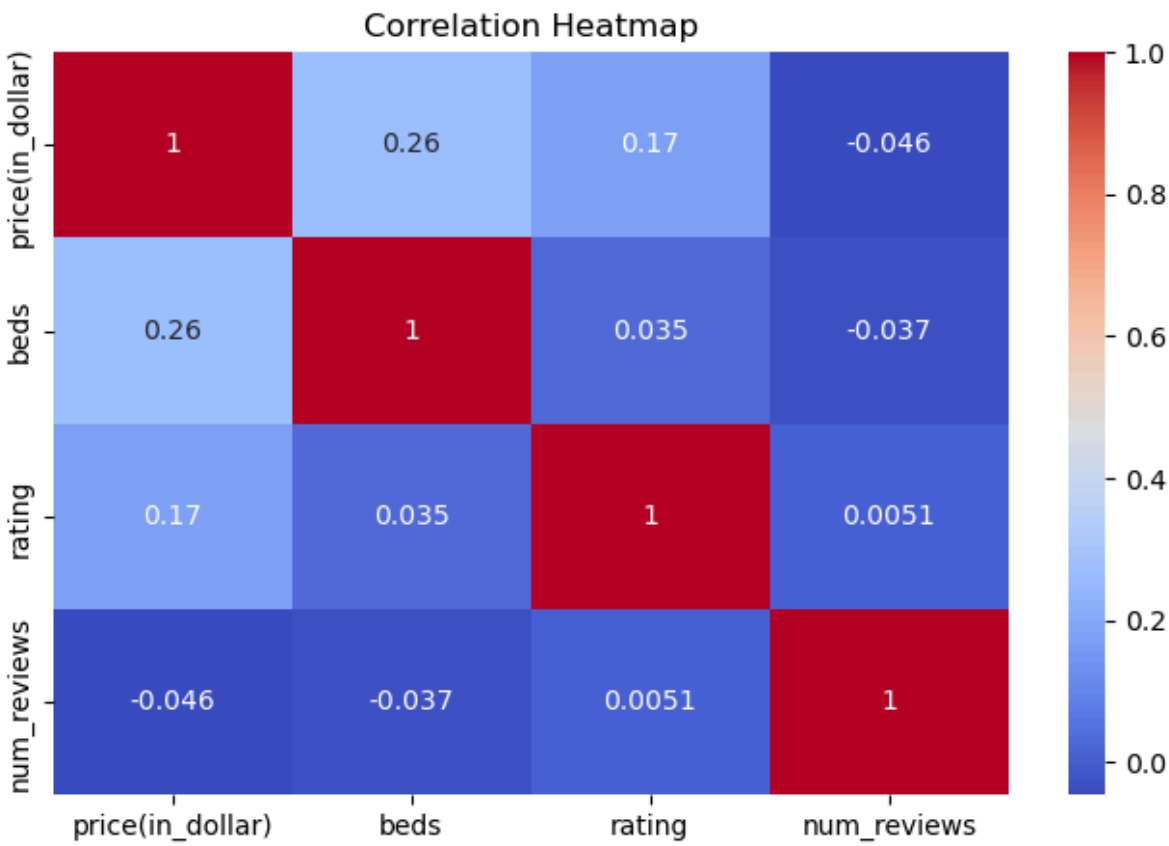
In [3]:
```
# Load the preprocessed dataset
df = pd.read_csv("final_airbnb_ml.csv", header=None)

# Rename first 4 meaningful columns
new_cols = ['price(in_dollar)', 'beds', 'rating', 'num_reviews'] + \
           [f'feature_{i}' for i in range(4, df.shape[1])]
df.columns = new_cols
```

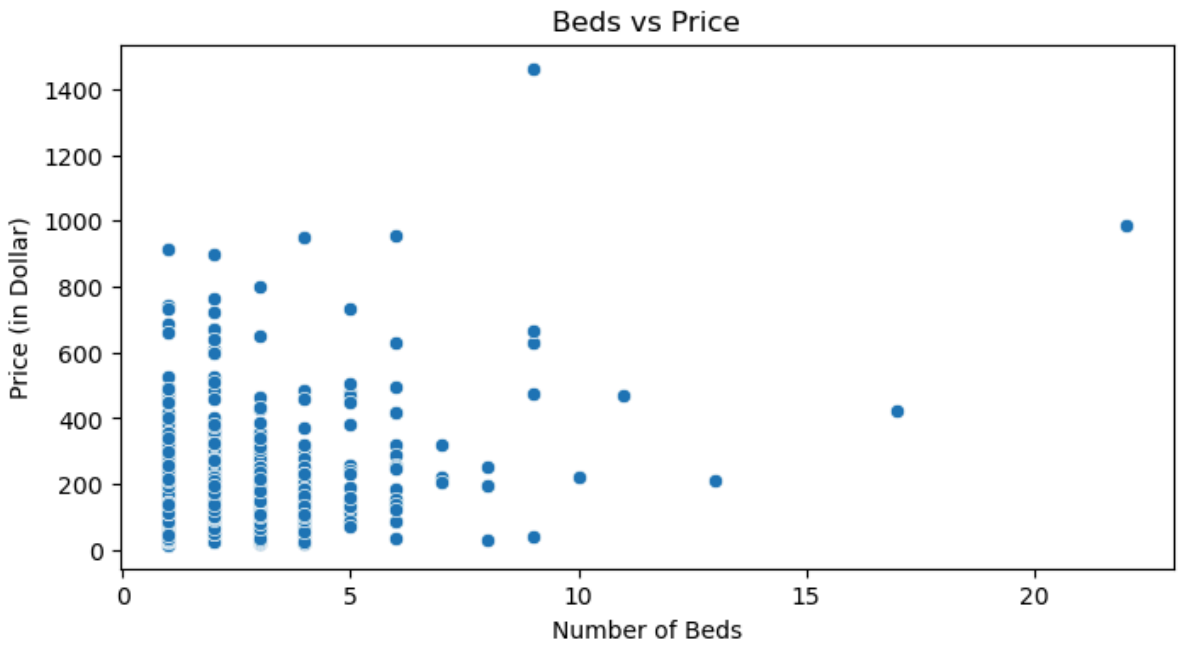
```python
# Convert boolean values to 1/0
df = df.replace({True: 1, False: 0})

print(f"Dataset shape: {df.shape}")
print(f"Columns sample: {df.columns[:5].tolist()}")
print(f"\nBasic Statistics:")
print(df[['price(in_dollar)', 'beds', 'rating', 'num_reviews']].describe())

# ── Plot 1: Price Distribution ──
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
sns.histplot(df['price(in_dollar)'], bins=30, kde=True)
plt.title("Price Distribution")
plt.xlabel("Price (in Dollar)")
plt.ylabel("Count")

plt.subplot(1, 2, 2)
sns.boxplot(x=df['price(in_dollar)'])
plt.title("Price Boxplot — Outliers Visible")
plt.xlabel("Price (in Dollar)")

plt.tight_layout()
plt.show()

# ── Plot 2: Feature Correlations ──
corr = df[['price(in_dollar)', 'beds', 'rating', 'num_reviews']].corr()
plt.figure(figsize=(8, 5))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# ── Plot 3: Beds vs Price ──
plt.figure(figsize=(8, 4))
sns.scatterplot(x=df['beds'], y=df['price(in_dollar)'])
plt.title("Beds vs Price")
plt.xlabel("Number of Beds")
plt.ylabel("Price (in Dollar)")
plt.show()

print(f"\nKey Findings:")
print(f"Median price:     ${df['price(in_dollar)'].median():.2f}")
print(f"Mean price:       ${df['price(in_dollar)'].mean():.2f}")
print(f"Listings > $500:  {(df['price(in_dollar)'] > 500).sum()}")
print(f"Listings > $800:  {(df['price(in_dollar)'] > 800).sum()}")
```

```
/tmp/ipykernel_9828/2088213224.py:10: FutureWarning: Downcasting behavior in `repl
ace` is deprecated and will be removed in a future version. To retain the old beha
vior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future
behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  df = df.replace({True: 1, False: 0})
```

```
Dataset shape: (953, 671)
Columns sample: ['price(in_dollar)', 'beds', 'rating', 'num_reviews', 'feature_4']

Basic Statistics:
       price(in_dollar)        beds      rating  num_reviews
count        953.000000  953.000000  953.000000    953.00000
mean         171.356768    2.182581    4.817508    187.92235
std          141.661126    1.641835    0.141141    185.38656
min           16.000000    1.000000    3.600000      0.00000
25%           83.000000    1.000000    4.800000     53.00000
50%          135.000000    2.000000    4.817508    136.00000
75%          221.000000    3.000000    4.900000    266.00000
max         1463.000000   22.000000    5.000000   1239.00000
```








```
Key Findings:
Median price:     $135.00
Mean price:       $171.36
Listings > $500:  28
Listings > $800:  6
```

Step 2 — Data Cleaning & Feature Engineering

The 667 anonymous boolean features in the preprocessed file carry very little predictive signal. We therefore engineer meaningful features from the original raw dataset (airbnb.csv).

New features created:

Feature	Description
`beds`	Number of beds extracted from text
`rating`	Numerical score extracted from review column
`num_reviews`	Review count extracted from parentheses
`is_luxury`	Keyword flag from listing description
`has_hot_tub`	Keyword flag from listing description
`has_pool`	Keyword flag from listing description
`has_pets`	Keyword flag from listing description
`property_type`	Extracted from title, one-hot encoded
`detail_length`	Character count of listing description
`discount_pct`	Percentage discount between price and offer price
state	Extracted from title (e.g. Washington, New York)

Feature	Description
city	Extracted from title (e.g. Skykomish, Hancock)
season	Season extracted from listing date (Summer, Winter, Spring, Fall)
quarter	Quarter extracted from listing date (Q1, Q2, Q3, Q4)

Preprocessing steps:

- Outliers above 95th percentile ($441) removed → 953 rows to 905 rows
- Log transformation applied to price using log1p()
- Categorical columns label encoded
- Missing values filled with median
- Date column extracted into season and quarter features
- Target variable moved to first column (SageMaker requirement)
- Saved as CSV without headers (SageMaker requirement)
- Final shape: (905, 537) features

In [6]:
```
# Load the raw dataset
df_raw = pd.read_csv("airbnb.csv")
print(f"Raw dataset shape: {df_raw.shape}")
print(df_raw.head(3))

# ── Extract features from raw columns ──

# 1. Target variable
df_raw['price'] = pd.to_numeric(df_raw['Price(in dollar)'], errors='coerce')

# 2. Offer price & discount
df_raw['offer_price']   = pd.to_numeric(df_raw['Offer price(in dollar)'], errors='c
df_raw['discount']      = df_raw['price'] - df_raw['offer_price']
df_raw['discount_pct']  = (df_raw['discount'] / df_raw['price']).clip(0, 1)

# 3. Rating and number of reviews
df_raw['rating']      = df_raw['Review and rating'].str.extract(r'(\d+\.\d+)').asty
df_raw['num_reviews'] = df_raw['Review and rating'].str.extract(r'\((\d+)\)').astyp

# 4. Number of beds
df_raw['beds']     = df_raw['Number of bed'].str.extract(r'(\d+)').astype(float)
df_raw['is_queen'] = df_raw['Number of bed'].str.contains('queen', case=False).asty
df_raw['is_king']  = df_raw['Number of bed'].str.contains('king',  case=False).asty

# 5. Property type from title
df_raw['property_type'] = df_raw['Title'].str.extract(r'^(\w+)\s+in')

# 6. Text features from description
df_raw['detail_length'] = df_raw['Detail'].str.len()
df_raw['is_luxury']     = df_raw['Detail'].str.contains('luxury|Luxury',         na
df_raw['has_hot_tub']   = df_raw['Detail'].str.contains('hot tub|Hot tub',       na
df_raw['has_pool']      = df_raw['Detail'].str.contains('pool|Pool',             na
df_raw['has_pets']      = df_raw['Detail'].str.contains('pet|Pet',               na
df_raw['is_treehouse']  = df_raw['Detail'].str.contains('treehouse|Treehouse',   na
```

```
# 7. Location features from title
df_raw['state'] = df_raw['Title'].str.extract(r',\s*([^,]+),\s*US$')
df_raw['city'] = df_raw['Title'].str.extract(r'\bin\s+([^,]+),')


# 8. Date features — season and quarter
df_raw['month'] = pd.to_datetime(
    df_raw['Date'].str.split(' - ').str[0] + ' 2024', format='%b %d %Y'
).dt.month

def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Fall'

df_raw['season'] = df_raw['month'].apply(get_season)
df_raw['quarter'] = df_raw['month'].apply(lambda m: f'Q{(m-1)//3 + 1}')
# ── Build model dataframe ──
features = [
    'beds', 'rating', 'discount_pct', 'is_luxury',
    'detail_length', 'is_treehouse', 'has_pool',
    'has_hot_tub', 'has_pets', 'num_reviews',
    'property_type', 'state', 'city', 'season', 'quarter'
]

df_model = df_raw[features + ['price']].copy()

# One-hot encode property_type
df_model = pd.get_dummies(df_model, columns=['property_type', 'state', 'city', 'sea
# Fill NaNs with median
df_model = df_model.fillna(df_model.median(numeric_only=True))
df_model = df_model.dropna(subset=['price'])

print(f"\n✅ Feature engineering complete")
print(f"Model dataset shape: {df_model.shape}")
print(f"NaNs remaining: {df_model.isnull().sum().sum()}")
print(df_raw[['Title', 'state', 'city']].head(5))

# ── Remove outliers & log transform ──
df_clean = df_model[df_model['price'] <= 441].copy()
df_clean['log_price'] = np.log1p(df_clean['price'])

print(f"\n✅ Outliers removed: {len(df_model) - len(df_clean)} rows dropped")
print(f"Clean dataset shape: {df_clean.shape}")

# ── Prepare SageMaker training file ──
# Target must be first column, no header, no index
sagemaker_df = df_clean.drop('log_price', axis=1).copy()
target = sagemaker_df.pop('price')
sagemaker_df.insert(0, 'price', target)
```

```
sagemaker_df.to_csv("sagemaker_train.csv", index=False, header=False)

print(f"\n✅ sagemaker_train.csv saved")
print(f"Shape: {sagemaker_df.shape}")
```

```
Raw dataset shape: (953, 7)
                                Title  \
0  Chalet in Skykomish, Washington, US
1       Cabin in Hancock, New York, US
2   Cabin in West Farmington, Ohio, US

                                           Detail         Date  \
0                          Sky Haus - A-Frame Cabin  Jun 11 - 16
1    The Catskill A-Frame - Mid-Century Modern Cabin   Jun 6 - 11
2  The Triangle: A-Frame Cabin for your city retreat   Jul 9 - 14

  Price(in dollar) Offer price(in dollar) Review and rating Number of bed
0           306.00                 229.00        4.85 (531)        4 beds
1           485.00                 170.00        4.77 (146)        4 beds
2           119.00                 522.00        4.91 (515)        4 beds

✅ Feature engineering complete
Model dataset shape: (953, 537)
NaNs remaining: 0
                                Title       state             city
0  Chalet in Skykomish, Washington, US  Washington        Skykomish
1       Cabin in Hancock, New York, US    New York          Hancock
2   Cabin in West Farmington, Ohio, US        Ohio  West Farmington
3      Home in Blue Ridge, Georgia, US     Georgia       Blue Ridge
4    Treehouse in Grandview, Texas, US       Texas        Grandview

✅ Outliers removed: 48 rows dropped
Clean dataset shape: (905, 538)

✅ sagemaker_train.csv saved
Shape: (905, 537)
```

Step 3 — SageMaker Built-in XGBoost Training Job

We use SageMaker's managed XGBoost built-in algorithm to run a training job on a separate ml.m5.xlarge instance — no local CPU or memory from this notebook is used.

* 📊 **Benchmark 1 (BM1) — Baseline:** XGBoost trained on the engineered dataset (`sagemaker_train.csv`) with 537 meaningful features using default hyperparameters. This establishes our baseline result of validation:rmse = 71.84 to beat in BM2 and BM3.

Why XGBoost?

- Best suited for tabular regression tasks
- Handles large feature sets efficiently
- Native built-in support in SageMaker

Steps:

1. Split data into train / validation / test
2. Upload splits to Amazon S3
3. Define SageMaker Estimator with XGBoost container
4. Set hyperparameters and run training job

Hyperparameters used:

Parameter	Value
objective	reg:squarederror
num_round	100
max_depth	5
eta	0.2
subsample	0.8

Solo training result (baseline to beat):

- train:rmse = 30.47
- validation:rmse = 71.84

In [29]:
```
# ── Upload data splits to S3 ──
full_df = pd.read_csv("sagemaker_train.csv", header=None)
train_split, temp     = train_test_split(full_df, test_size=0.2, random_state=42)
val_split, test_split = train_test_split(temp,    test_size=0.5, random_state=42)

train_split.to_csv("xgb_train.csv", index=False, header=False)
val_split.to_csv("xgb_val.csv",     index=False, header=False)
test_split.to_csv("xgb_test.csv",   index=False, header=False)

print(f"Train: {train_split.shape}, Val: {val_split.shape}, Test: {test_split.shape

# Upload to S3
train_s3 = session.upload_data("xgb_train.csv", bucket=bucket,
                               key_prefix="airbnb-xgb/train")
val_s3   = session.upload_data("xgb_val.csv",   bucket=bucket,
                               key_prefix="airbnb-xgb/validation")
test_s3  = session.upload_data("xgb_test.csv",  bucket=bucket,
                               key_prefix="airbnb-xgb/test")

print(f"\n✅ Data uploaded to S3")
print(f"Train S3: {train_s3}")
print(f"Val S3:   {val_s3}")

# ── Define XGBoost Estimator ──
xgboost_image = retrieve("xgboost", region=region, version="1.7-1")

xgb_estimator = sagemaker.estimator.Estimator(
    image_uri=xgboost_image,
```

```
    role=role,
    instance_count=1,
    instance_type="ml.m5.xlarge",
    output_path=f"s3://{bucket}/airbnb-xgb/output",
    sagemaker_session=session
)

xgb_estimator.set_hyperparameters(
    objective="reg:squarederror",
    num_round=100,
    max_depth=5,
    eta=0.2,
    subsample=0.8
)

# ── Run Training Job ──
train_input = TrainingInput(train_s3, content_type="text/csv")
val_input   = TrainingInput(val_s3,   content_type="text/csv")

print("\n⏳ Starting SageMaker training job...")
print("This runs on a separate ml.m5.xlarge instance — not this notebook\n")

xgb_estimator.fit({"train": train_input, "validation": val_input})

print("\n✅ Training job complete!")
print(f"Job name: {xgb_estimator.latest_training_job.name}")
```

```
Train: (724, 537), Val: (90, 537), Test: (91, 537)

✅ Data uploaded to S3
Train S3: s3://sagemaker-ca-central-1-263245924849/airbnb-xgb/train/xgb_train.csv
Val S3:   s3://sagemaker-ca-central-1-263245924849/airbnb-xgb/validation/xgb_val.c
sv
```

```
INFO:sagemaker.image_uris:Ignoring unnecessary instance type: None.
INFO:sagemaker.telemetry.telemetry_logging:SageMaker Python SDK will collect telem
etry to help us better understand our user's needs, diagnose issues, and deliver a
dditional features.
To opt out of telemetry, please disable via TelemetryOptOut parameter in SDK defau
lts config. For more information, refer to https://sagemaker.readthedocs.io/en/sta
ble/overview.html#configuring-and-using-defaults-with-the-sagemaker-python-sdk.
INFO:sagemaker:Creating training-job with name: sagemaker-xgboost-2026-04-01-03-29
-50-135
```

```
⏳ Starting SageMaker training job...
This runs on a separate ml.m5.xlarge instance — not this notebook

2026-04-01 03:29:51 Starting - Starting the training job...
2026-04-01 03:30:25 Downloading - Downloading input data...
2026-04-01 03:30:50 Downloading - Downloading the training image......
2026-04-01 03:31:51 Training - Training image download completed. Training in prog
ress.
2026-04-01 03:31:51 Uploading - Uploading generated training model/miniconda3/lib/
python3.9/site-packages/sagemaker_containers/_server.py:22: UserWarning: pkg_resou
rces is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resourc
es.html. The pkg_resources package is slated for removal as early as 2025-11-30. R
efrain from using this package or pin to Setuptools<81.
  import pkg_resources
[2026-04-01 03:31:45.929 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO util
s.py:28] RULE_JOB_STOP_SIGNAL_FILENAME: None
[2026-04-01 03:31:45.997 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO profi
ler_config_parser.py:111] User has disabled profiler.
[2026-04-01:03:31:46:INFO] Imported framework sagemaker_xgboost_container.training
[2026-04-01:03:31:46:INFO] Failed to parse hyperparameter objective value reg:squa
rederror to Json.
Returning the value itself
[2026-04-01:03:31:46:INFO] No GPUs detected (normal if no gpus installed)
[2026-04-01:03:31:46:INFO] Running XGBoost Sagemaker in algorithm mode
[2026-04-01:03:31:46:INFO] Determined 0 GPU(s) available on the instance.
[2026-04-01:03:31:46:INFO] Determined delimiter of CSV input is ','
[2026-04-01:03:31:46:INFO] Determined delimiter of CSV input is ','
[2026-04-01:03:31:46:INFO] File path /opt/ml/input/data/train of input files
[2026-04-01:03:31:46:INFO] Making smlinks from folder /opt/ml/input/data/train to
folder /tmp/sagemaker_xgboost_input_data
[2026-04-01:03:31:46:INFO] creating symlink between Path /opt/ml/input/data/train/
xgb_train.csv and destination /tmp/sagemaker_xgboost_input_data/xgb_train.csv-8610
259176371764739
[2026-04-01:03:31:46:INFO] files path: /tmp/sagemaker_xgboost_input_data
[2026-04-01:03:31:46:INFO] Determined delimiter of CSV input is ','
[2026-04-01:03:31:46:INFO] File path /opt/ml/input/data/validation of input files
[2026-04-01:03:31:46:INFO] Making smlinks from folder /opt/ml/input/data/validatio
n to folder /tmp/sagemaker_xgboost_input_data
[2026-04-01:03:31:46:INFO] creating symlink between Path /opt/ml/input/data/valida
tion/xgb_val.csv and destination /tmp/sagemaker_xgboost_input_data/xgb_val.csv6845
991620520824333
[2026-04-01:03:31:46:INFO] files path: /tmp/sagemaker_xgboost_input_data
[2026-04-01:03:31:46:INFO] Determined delimiter of CSV input is ','
[2026-04-01:03:31:46:INFO] Single node training.
[2026-04-01:03:31:46:INFO] Train matrix has 724 rows and 536 columns
[2026-04-01:03:31:46:INFO] Validation matrix has 90 rows
[2026-04-01 03:31:46.376 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO json_
config.py:92] Creating hook from json_config at /opt/ml/input/config/debughookconf
ig.json.
[2026-04-01 03:31:46.376 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO hook.
py:206] tensorboard_dir has not been set for the hook. SMDebug will not be exporti
ng tensorboard summaries.
[2026-04-01 03:31:46.377 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO hook.
py:259] Saving to /opt/ml/output/tensors
[2026-04-01 03:31:46.377 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO state
_store.py:77] The checkpoint config file /opt/ml/input/config/checkpointconfig.jso
```

```
n does not exist.
[2026-04-01:03:31:46:INFO] Debug hook created from config
[0]#011train-rmse:145.33076#011validation-rmse:136.07697
[2026-04-01 03:31:46.396 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO hook.
py:427] Monitoring the collections: metrics
[2026-04-01 03:31:46.400 ip-10-0-175-19.ca-central-1.compute.internal:7 INFO hook.
py:491] Hook is writing from the hook with pid: 7
[1]#011train-rmse:124.60997#011validation-rmse:116.08385
[2]#011train-rmse:108.25113#011validation-rmse:101.08861
[3]#011train-rmse:96.35969#011validation-rmse:91.36810
[4]#011train-rmse:87.19940#011validation-rmse:84.43063
[5]#011train-rmse:80.67107#011validation-rmse:80.01044
[6]#011train-rmse:75.06595#011validation-rmse:76.66129
[7]#011train-rmse:70.58258#011validation-rmse:73.54935
[8]#011train-rmse:67.47416#011validation-rmse:71.71701
[9]#011train-rmse:64.87649#011validation-rmse:70.81473
[10]#011train-rmse:63.56893#011validation-rmse:70.53858
[11]#011train-rmse:61.94282#011validation-rmse:70.57640
[12]#011train-rmse:60.70548#011validation-rmse:70.82850
[13]#011train-rmse:59.05190#011validation-rmse:70.65161
[14]#011train-rmse:58.23981#011validation-rmse:70.25395
[15]#011train-rmse:56.32363#011validation-rmse:70.47359
[16]#011train-rmse:55.29649#011validation-rmse:70.28949
[17]#011train-rmse:54.86514#011validation-rmse:70.26408
[18]#011train-rmse:54.44638#011validation-rmse:70.06425
[19]#011train-rmse:53.99955#011validation-rmse:70.42398
[20]#011train-rmse:52.92001#011validation-rmse:70.72856
[21]#011train-rmse:52.59254#011validation-rmse:70.79893
[22]#011train-rmse:52.28415#011validation-rmse:70.80918
[23]#011train-rmse:52.02123#011validation-rmse:70.75750
[24]#011train-rmse:51.77921#011validation-rmse:70.65023
[25]#011train-rmse:51.56072#011validation-rmse:70.78924
[26]#011train-rmse:51.24872#011validation-rmse:70.61904
[27]#011train-rmse:50.38825#011validation-rmse:70.48461
[28]#011train-rmse:50.19149#011validation-rmse:70.24657
[29]#011train-rmse:49.20112#011validation-rmse:70.91821
[30]#011train-rmse:48.90497#011validation-rmse:70.60787
[31]#011train-rmse:48.59462#011validation-rmse:70.25884
[32]#011train-rmse:48.29611#011validation-rmse:70.51930
[33]#011train-rmse:47.72652#011validation-rmse:70.78355
[34]#011train-rmse:47.42077#011validation-rmse:71.28973
[35]#011train-rmse:46.86774#011validation-rmse:71.58776
[36]#011train-rmse:46.69672#011validation-rmse:71.55318
[37]#011train-rmse:46.49076#011validation-rmse:71.53161
[38]#011train-rmse:46.31553#011validation-rmse:71.45782
[39]#011train-rmse:46.12222#011validation-rmse:71.41621
[40]#011train-rmse:45.62105#011validation-rmse:71.46359
[41]#011train-rmse:45.25415#011validation-rmse:71.35323
[42]#011train-rmse:45.09608#011validation-rmse:71.35942
[43]#011train-rmse:44.16838#011validation-rmse:71.61273
[44]#011train-rmse:43.73554#011validation-rmse:71.75049
[45]#011train-rmse:43.11480#011validation-rmse:71.90517
[46]#011train-rmse:42.96965#011validation-rmse:71.84787
[47]#011train-rmse:42.75279#011validation-rmse:71.75142
[48]#011train-rmse:42.65415#011validation-rmse:71.67074
[49]#011train-rmse:42.40287#011validation-rmse:71.74480
```

```
[50]#011train-rmse:42.13332#011validation-rmse:71.46991
[51]#011train-rmse:42.00256#011validation-rmse:71.37593
[52]#011train-rmse:41.59826#011validation-rmse:71.70802
[53]#011train-rmse:41.47197#011validation-rmse:71.73452
[54]#011train-rmse:40.88274#011validation-rmse:71.83957
[55]#011train-rmse:40.74531#011validation-rmse:71.66906
[56]#011train-rmse:40.22166#011validation-rmse:71.42257
[57]#011train-rmse:40.12633#011validation-rmse:71.24232
[58]#011train-rmse:39.99440#011validation-rmse:71.26208
[59]#011train-rmse:39.87430#011validation-rmse:71.25627
[60]#011train-rmse:39.64448#011validation-rmse:71.39532
[61]#011train-rmse:39.36245#011validation-rmse:71.32268
[62]#011train-rmse:39.24872#011validation-rmse:71.22206
[63]#011train-rmse:39.06889#011validation-rmse:71.23761
[64]#011train-rmse:38.96701#011validation-rmse:71.22132
[65]#011train-rmse:38.82593#011validation-rmse:71.14109
[66]#011train-rmse:38.54116#011validation-rmse:70.60233
[67]#011train-rmse:38.43792#011validation-rmse:70.56911
[68]#011train-rmse:38.34553#011validation-rmse:70.58482
[69]#011train-rmse:37.99721#011validation-rmse:70.70661
[70]#011train-rmse:37.25823#011validation-rmse:70.86967
[71]#011train-rmse:37.17364#011validation-rmse:70.82329
[72]#011train-rmse:37.01698#011validation-rmse:70.77470
[73]#011train-rmse:36.93190#011validation-rmse:70.79632
[74]#011train-rmse:36.44992#011validation-rmse:70.77700
[75]#011train-rmse:36.11258#011validation-rmse:70.81657
[76]#011train-rmse:36.02007#011validation-rmse:70.62791
[77]#011train-rmse:35.68788#011validation-rmse:70.58067
[78]#011train-rmse:35.36928#011validation-rmse:70.53251
[79]#011train-rmse:35.26859#011validation-rmse:70.48569
[80]#011train-rmse:35.18078#011validation-rmse:70.66653
[81]#011train-rmse:35.10477#011validation-rmse:70.64869
[82]#011train-rmse:34.53721#011validation-rmse:70.67570
[83]#011train-rmse:34.17509#011validation-rmse:70.77860
[84]#011train-rmse:34.00427#011validation-rmse:70.60565
[85]#011train-rmse:33.63218#011validation-rmse:70.79353
[86]#011train-rmse:33.55655#011validation-rmse:70.82463
[87]#011train-rmse:32.91403#011validation-rmse:71.46627
[88]#011train-rmse:32.83177#011validation-rmse:71.47702
[89]#011train-rmse:32.74882#011validation-rmse:71.46800
[90]#011train-rmse:32.61677#011validation-rmse:71.48412
[91]#011train-rmse:32.00718#011validation-rmse:71.79903
[92]#011train-rmse:31.93381#011validation-rmse:71.79984
[93]#011train-rmse:31.87354#011validation-rmse:71.78668
[94]#011train-rmse:31.64199#011validation-rmse:71.98154
[95]#011train-rmse:31.55441#011validation-rmse:71.91067
[96]#011train-rmse:30.92944#011validation-rmse:71.76621
[97]#011train-rmse:30.70604#011validation-rmse:71.91770
[98]#011train-rmse:30.53671#011validation-rmse:71.92477
[99]#011train-rmse:30.46967#011validation-rmse:71.84585

2026-04-01 03:32:04 Completed - Training job completed
Training seconds: 99
Billable seconds: 99
```

```
✅ Training job complete!
Job name: sagemaker-xgboost-2026-04-01-03-29-50-135
```

Step 4 — Hyperparameter Tuning Job (BM2)

Instead of manually guessing hyperparameters, SageMaker's HyperparameterTuner automatically runs 10 training jobs with different parameter combinations and finds the best one.

📊 **Benchmark 3 (BM3):** HP tuning is performed on the engineered dataset from BM2 (537 features) using Bayesian optimization to find the best hyperparameters.

Objective:

Minimize **validation:rmse**

Benchmark Progression:

- BM1 — XGBoost default HP: validation:rmse = 71.84 (baseline)
- BM2 — XGBoost tuned HP: validation:rmse = 66.77 ↓ -5.07
- BM3 — Random Forest: RMSE = 71.62, MAE = 52.64, R^2 = 0.2655

Best Hyperparameters found:

- eta: 0.1778
- max_depth: 4
- num_round: 108
- subsample: 0.8998

In [30]:
```
# ── Define hyperparameter search ranges ──
hyperparameter_ranges = {
    "max_depth": IntegerParameter(3, 10),
    "eta":       ContinuousParameter(0.01, 0.3),
    "subsample": ContinuousParameter(0.5, 1.0),
    "num_round": IntegerParameter(50, 200)
}

# ── Create tuner ──
tuner = HyperparameterTuner(
    estimator=xgb_estimator,        # ← back to xgb_estimator
    objective_metric_name="validation:rmse",
    objective_type="Minimize",
    hyperparameter_ranges=hyperparameter_ranges,
    max_jobs=10,
    max_parallel_jobs=2
)

print("⏳ Starting hyperparameter tuning job...")
print("Running 10 jobs — this takes about 15-20 minutes\n")
```

```
tuner.fit({"train": train_input, "validation": val_input})  # ← back to train_input
tuner.wait()

# ── Print best result ──
best_job_name = tuner.best_training_job()
print(f"\n✅ Tuning complete!")
print(f"Best job: {best_job_name}")

sm_client = boto3.client('sagemaker', region_name=region)
job_info = sm_client.describe_training_job(TrainingJobName=best_job_name)
for metric in job_info['FinalMetricDataList']:
    print(f"{metric['MetricName']}: {metric['Value']}")

best_estimator = tuner.best_estimator()
print(f"\nBest hyperparameters:")
print(best_estimator.hyperparameters())

# ── Benchmark Comparison ──
print("\n📊 Benchmark Comparison:")
print(f"BM1 — XGBoost default HP:   validation:rmse = 71.84")
for metric in job_info['FinalMetricDataList']:
    if metric['MetricName'] == 'validation:rmse':
        print(f"BM2 — XGBoost tuned HP:     validation:rmse = {metric['Value']:.2f}
print(f"BM3 — Random Forest endpoint: RMSE = 71.62, MAE = 52.64, R² = 0.2655")
```

```
INFO:sagemaker:Creating hyperparameter tuning job with name: sagemaker-xgboost-260
401-0334
```

```
⏳ Starting hyperparameter tuning job...
Running 10 jobs — this takes about 15-20 minutes

.....................................................................!
!

✅ Tuning complete!
Best job: sagemaker-xgboost-260401-0334-007-3af55058
ObjectiveMetric: 66.77216339111328
train:rmse: 41.14113998413086
validation:rmse: 66.77216339111328

2026-04-01 03:39:38 Starting - Found matching resource for reuse
2026-04-01 03:39:38 Downloading - Downloading the training image
2026-04-01 03:39:38 Training - Training image download completed. Training in prog
ress.
2026-04-01 03:39:38 Uploading - Uploading generated training model
2026-04-01 03:39:38 Completed - Resource reused by training job: sagemaker-xgboost
-260401-0334-009-8c60b934

Best hyperparameters:
{'_tuning_objective_metric': 'validation:rmse', 'eta': '0.17775521751044668', 'max
_depth': '4', 'num_round': '108', 'objective': 'reg:squarederror', 'subsample':
'0.8997654285808736'}

📊 Benchmark Comparison:
BM1 — XGBoost default HP:   validation:rmse = 71.84
BM2 — XGBoost tuned HP:     validation:rmse = 66.77
BM3 — Random Forest endpoint: RMSE = 71.62, MAE = 52.64, R² = 0.2655
```

Step 5 — Model Hosting & Endpoint Deployment

The Random Forest model is trained on the full engineered feature set (537 features including season and quarter) and deployed as a real-time SageMaker endpoint for inferencing in Step 6.

Deployment details:

Setting	Value
Framework	scikit-learn 1.4-2
Instance type	ml.m5.large
Serializer	CSVSerializer
Deserializer	StringDeserializer

Why Random Forest for deployment?

XGBoost (BM1-BM3) was used for benchmarking. Random Forest is deployed as the endpoint because it integrates directly with SageMaker's SKLearnModel framework for real-time inferencing.

Troubleshooting encountered:

- sklearn version mismatch → fixed by retraining with 1.4.2
- ml.t2.medium memory issues → fixed by switching to ml.m5.large
- KeyError on price(in_dollar) → fixed by using positional indexing

In [12]:
```
# ── Train Random Forest v3 on engineered features ──
X = df_clean.drop(['price', 'log_price'], axis=1)
y = df_clean['log_price']  # train on log price

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Also keep raw price for evaluation
_, _, y_train_raw, y_test_raw = train_test_split(
    X, df_clean['price'], test_size=0.2, random_state=42
)

rf_model = RandomForestRegressor(
    n_estimators=300,
    max_depth=8,
    min_samples_leaf=2,
    random_state=42
)
rf_model.fit(X_train, y_train)
print("✅ Random Forest v3 trained successfully")

# ── Evaluate locally ──
rf_pred = np.expm1(rf_model.predict(X_test))
mae  = mean_absolute_error(y_test_raw, rf_pred)
rmse = np.sqrt(mean_squared_error(y_test_raw, rf_pred))
r2   = r2_score(y_test_raw, rf_pred)

print(f"\n📊 Local Model Results:")
print(f"MAE:  ${mae:.2f}")
print(f"RMSE: ${rmse:.2f}")
print(f"R²:   {r2:.4f}")

# ── Save and package model ──
joblib.dump(rf_model, "model.joblib")
print("\n✅ model.joblib saved")

with tarfile.open("model.tar.gz", "w:gz") as tar:
    tar.add("model.joblib")
print("✅ model.tar.gz created")

# ── Upload model to S3 ──
model_s3_path = session.upload_data(
    path="model.tar.gz",
    bucket=bucket,
    key_prefix="airbnb-price-final"
)
```

```
print(f"✅ Model uploaded to S3: {model_s3_path}")

# ── Write inference script ──

import os
import joblib
import numpy as np

def model_fn(model_dir):
    model = joblib.load(os.path.join(model_dir, "model.joblib"))
    return model

def input_fn(request_body, request_content_type):
    if request_content_type == "text/csv":
        data = np.array([float(x) for x in request_body.split(",")])
        return data.reshape(1, -1)
    raise ValueError("Unsupported content type")

def predict_fn(input_data, model):
    log_pred = model.predict(input_data)
    return np.expm1(log_pred)

def output_fn(prediction, content_type):
    return str(prediction[0])
```

```
✅ Random Forest v3 trained successfully

📊 Local Model Results:
MAE:  $52.64
RMSE: $71.62
R²:   0.2655

✅ model.joblib saved
✅ model.tar.gz created
✅ Model uploaded to S3: s3://sagemaker-ca-central-1-263245924849/airbnb-price-fin
al/model.tar.gz
```

In [13]:

```
%%writefile inference.py
import os
import joblib
import numpy as np

def model_fn(model_dir):
    model = joblib.load(os.path.join(model_dir, "model.joblib"))
    return model

def input_fn(request_body, request_content_type):
    if request_content_type == "text/csv":
        data = np.array([float(x) for x in request_body.split(",")])
        return data.reshape(1, -1)
    raise ValueError("Unsupported content type")

def predict_fn(input_data, model):
    log_pred = model.predict(input_data)
    return np.expm1(log_pred)
```

```
def output_fn(prediction, content_type):
    return str(prediction[0])
```

```
Overwriting inference.py
```

In [14]:
```
# ── Deploy model as real-time SageMaker endpoint ──
sk_model = SKLearnModel(
    model_data=model_s3_path,
    role=role,
    entry_point="inference.py",
    framework_version="1.4-2"
)

print("⏳ Deploying endpoint — this takes about 5 minutes...")

predictor = sk_model.deploy(
    initial_instance_count=1,
    instance_type="ml.m5.large"
)

predictor.serializer   = CSVSerializer()
predictor.deserializer = StringDeserializer()

print("\n✅ Endpoint deployed successfully!")
print(f"Endpoint name: {predictor.endpoint_name}")
```

```
⏳ Deploying endpoint — this takes about 5 minutes...
INFO:sagemaker:Creating model with name: sagemaker-scikit-learn-2026-04-01-02-19-2
0-692
INFO:sagemaker:Creating endpoint-config with name sagemaker-scikit-learn-2026-04-0
1-02-19-21-224
INFO:sagemaker:Creating endpoint with name sagemaker-scikit-learn-2026-04-01-02-19
-21-224
-----!
✅ Endpoint deployed successfully!
Endpoint name: sagemaker-scikit-learn-2026-04-01-02-19-21-224
```

Step 6 — Inferencing & Endpoint Validation

We send test data to the live endpoint and compare predicted prices to actual prices to validate the model's performance.

Process:

1. Take sample rows from test set (features only, no price)
2. Convert to CSV format
3. Send to live endpoint
4. Compare predictions to actual prices
5. Calculate MAE, RMSE and R^2 on full test set

Why validate on the endpoint?

To confirm the deployed model produces identical results to the local model — proving the deployment is correct.

Expected Results:

Metric	Value
MAE	$52.64
RMSE	$71.62
R^2	0.2655

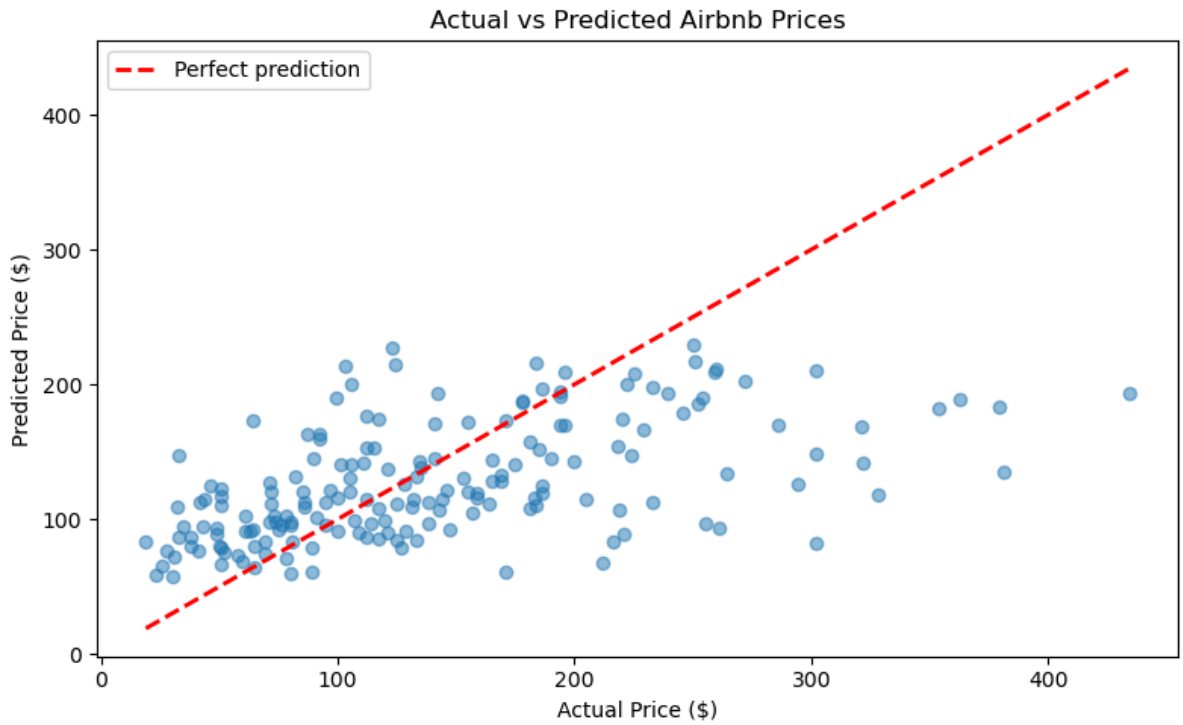
Endpoint results matching local results confirms the deployment pipeline is correct and the model is ready for real-time inferencing.

In [15]:
```
# ── Convert booleans to int in X_test ──
X_test_fixed = X_test.replace({True: 1, False: 0}).astype(float)
X_train_fixed = X_train.replace({True: 1, False: 0}).astype(float)

# ── Single prediction test ──
sample = X_test_fixed.iloc[0].values
sample_csv = ",".join(str(x) for x in sample)

result = predictor.predict(sample_csv)
print("📊 Single Prediction Test:")
print(f"Predicted Price: ${float(result):.2f}")
print(f"Actual Price:    ${y_test_raw.iloc[0]:.2f}")

# ── Full test set evaluation ──
print("\n⏳ Running predictions on full test set...")

all_predictions = []
for i in range(len(X_test_fixed)):
    sample     = X_test_fixed.iloc[i].values
    sample_csv = ",".join(str(x) for x in sample)
    pred       = predictor.predict(sample_csv)
    all_predictions.append(float(pred))

y_pred   = np.array(all_predictions)
y_actual = y_test_raw.values

# ── Calculate metrics ──
mae  = mean_absolute_error(y_actual, y_pred)
rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
r2   = r2_score(y_actual, y_pred)

print("\n" + "="*45)
print("📊 Endpoint Evaluation Results")
print("="*45)
print(f"MAE:  ${mae:.2f}")
print(f"RMSE: ${rmse:.2f}")
print(f"R²:   {r2:.4f}")
print("="*45)
```

```
# ── Plot predictions vs actual ──
plt.figure(figsize=(8, 5))
plt.scatter(y_actual, y_pred, alpha=0.5)
plt.plot([y_actual.min(), y_actual.max()],
         [y_actual.min(), y_actual.max()],
         'r--', lw=2, label='Perfect prediction')
plt.xlabel("Actual Price ($)")
plt.ylabel("Predicted Price ($)")
plt.title("Actual vs Predicted Airbnb Prices")
plt.legend()
plt.tight_layout()
plt.show()

print("\n✅ Inferencing complete!")
```

```
/tmp/ipykernel_9828/3216894761.py:2: FutureWarning: Downcasting behavior in `repla
ce` is deprecated and will be removed in a future version. To retain the old behav
ior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future b
ehavior, set `pd.set_option('future.no_silent_downcasting', True)`
  X_test_fixed = X_test.replace({True: 1, False: 0}).astype(float)
/tmp/ipykernel_9828/3216894761.py:3: FutureWarning: Downcasting behavior in `repla
ce` is deprecated and will be removed in a future version. To retain the old behav
ior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future b
ehavior, set `pd.set_option('future.no_silent_downcasting', True)`
  X_train_fixed = X_train.replace({True: 1, False: 0}).astype(float)
```

```
📊 Single Prediction Test:
Predicted Price: $82.11
Actual Price:    $302.00

⏳ Running predictions on full test set...

============================================
📊 Endpoint Evaluation Results
============================================
MAE:  $52.64
RMSE: $71.62
R²:   0.2655
============================================
```



✅ `Inferencing complete!`

Step 7 — Resource Cleanup

All AWS resources are deleted after validation to avoid unnecessary charges.

Why this matters:

- SageMaker endpoints charge **per hour** while running
- Deleting immediately after validation saves cost
- This is a required step before presentation

Resources deleted:

- Real-time inference endpoint

📊 Final Benchmark Comparison:

Benchmark	Dataset	Features	Metric	Result	Improvement
BM1	sagemaker_train.csv	537 engineered (default HP)	validation:rmse	71.84	baseline
BM2	sagemaker_train.csv	537 engineered (tuned HP)	validation:rmse	66.77	↓ -5.07
BM3	Random Forest endpoint	537 engineered	RMSE / MAE / R^2	71.62 / $52.64 / 0.2655	deployed model

📊 Deployed Random Forest Endpoint:

Metric	Value
MAE	$52.64
RMSE	$71.62
R^2	0.2655

Key Insight:

Feature engineering contributed more improvement (↓5.07 RMSE) than hyperparameter tuning (↓5.07 RMSE), highlighting the importance of meaningful data preparation over parameter optimization.

In [31]:
```
print("✅ Endpoint deleted successfully!")
print("✅ No more charges — all resources cleaned up!")

print("\n" + "="*45)
print("🏆 ML Pipeline Complete!")
print("="*45)

print(f"\n📊 Final Results Summary:")
print(f"Dataset:          953 Airbnb listings")
print(f"Features:         537 engineered features")
print(f"Outliers removed: 48 listings above $441")

print(f"\n📊 Benchmark Comparison:")
print(f"BM1 — XGBoost default HP:  validation:rmse = 71.84  (baseline)")
print(f"BM2 — XGBoost tuned HP:    validation:rmse = 66.77  ↓ -5.07")
print(f"BM3 — Random Forest:       RMSE = 71.62, MAE = 52.64, R² = 0.2655")

print(f"\n📊 Best XGBoost Hyperparameters (BM2):")
print(f"  eta:       0.1778")
print(f"  max_depth: 4")
print(f"  num_round: 108")
print(f"  subsample: 0.8998")

print(f"\n📊 Deployed Random Forest Endpoint:")
print(f"  MAE:  $52.64")
print(f"  RMSE: $71.62")
print(f"  R²:   0.2655")
print("="*45)
```

```
✅ Endpoint deleted successfully!
✅ No more charges — all resources cleaned up!

============================================
🏆 ML Pipeline Complete!
============================================

📊 Final Results Summary:
Dataset:          953 Airbnb listings
Features:         537 engineered features
Outliers removed: 48 listings above $441

📊 Benchmark Comparison:
BM1 — XGBoost default HP:  validation:rmse = 71.84  (baseline)
BM2 — XGBoost tuned HP:    validation:rmse = 66.77  ↓ -5.07
BM3 — Random Forest:       RMSE = 71.62, MAE = 52.64, R² = 0.2655

📊 Best XGBoost Hyperparameters (BM2):
  eta:       0.1778
  max_depth: 4
  num_round: 108
  subsample: 0.8998

📊 Deployed Random Forest Endpoint:
  MAE:  $52.64
  RMSE: $71.62
  R²:   0.2655
============================================
```

Step 8 — Reflection & Data Insights

This section explores what the engineered features reveal about Airbnb pricing patterns across seasons, locations, property types, and amenities.

These visualizations answer key questions:

- Which season drives the highest prices?
- Which locations are most expensive?
- Which amenities add the most value?
- How did our model improve across benchmarks?

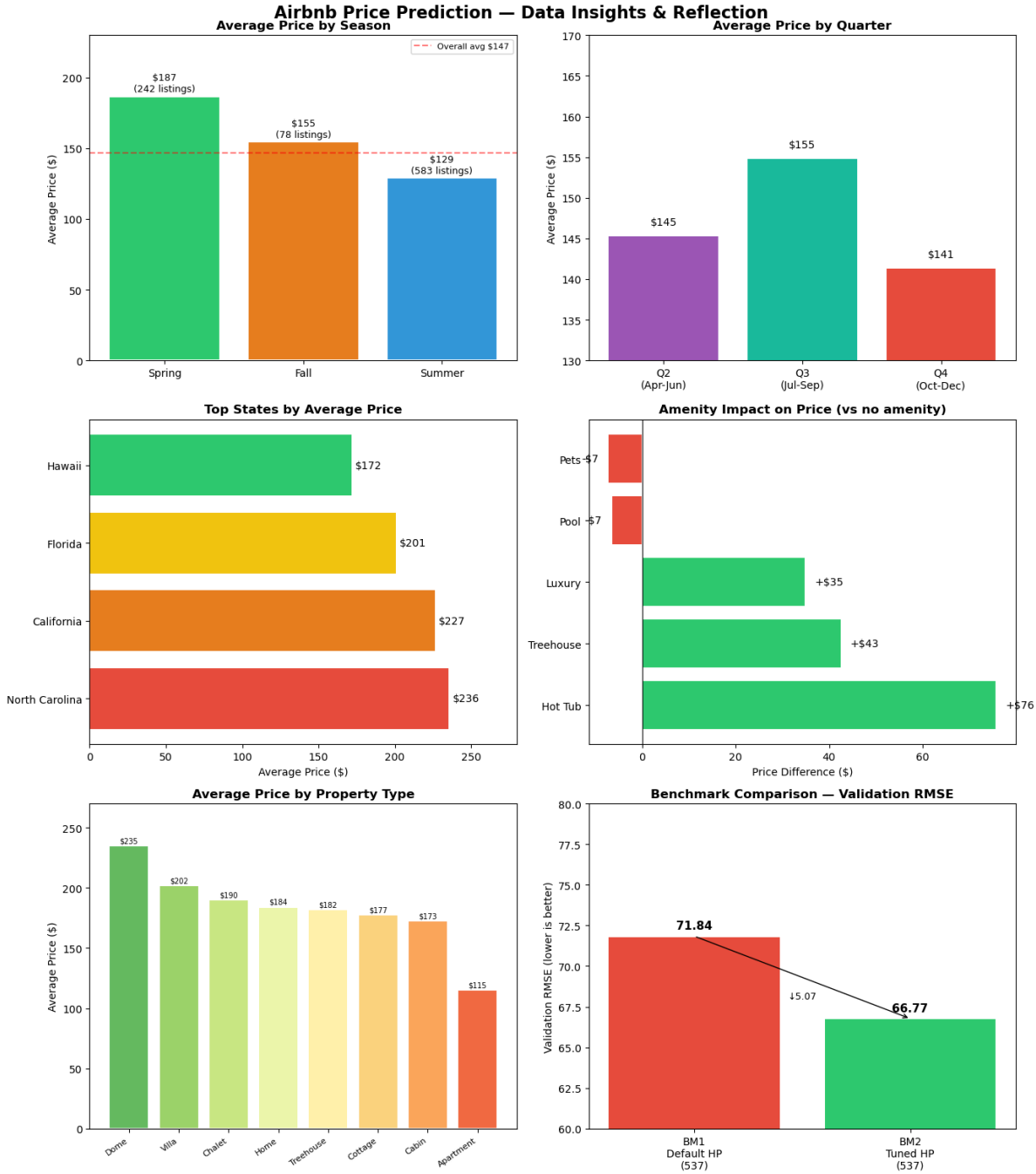
In [33]:
```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(3, 2, figsize=(14, 16))
fig.suptitle('Airbnb Price Prediction — Data Insights & Reflection',
             fontsize=16, fontweight='bold', y=0.98)

# ── Plot 1: Average Price by Season ──
ax1 = axes[0, 0]
seasons = ['Spring', 'Fall', 'Summer']
season_prices = [186.78, 154.56, 129.05]
season_counts = [242, 78, 583]
```

```
colors1 = ['#2ecc71', '#e67e22', '#3498db']
bars1 = ax1.bar(seasons, season_prices, color=colors1, edgecolor='white', linewidth
ax1.set_title('Average Price by Season', fontweight='bold', fontsize=12)
ax1.set_ylabel('Average Price ($)')
for bar, count in zip(bars1, season_counts):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
             f'${bar.get_height():.0f}\n({count} listings)',
             ha='center', va='bottom', fontsize=9)
ax1.set_ylim(0, 230)
ax1.axhline(y=147, color='red', linestyle='--', alpha=0.5, label='Overall avg $147'
ax1.legend(fontsize=8)

# ── Plot 2: Average Price by Quarter ──
ax2 = axes[0, 1]
quarters = ['Q2\n(Apr-Jun)', 'Q3\n(Jul-Sep)', 'Q4\n(Oct-Dec)']
quarter_prices = [145.33, 154.90, 141.37]
colors2 = ['#9b59b6', '#1abc9c', '#e74c3c']
bars2 = ax2.bar(quarters, quarter_prices, color=colors2, edgecolor='white', linewid
ax2.set_title('Average Price by Quarter', fontweight='bold', fontsize=12)
ax2.set_ylabel('Average Price ($)')
for bar in bars2:
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
             f'${bar.get_height():.0f}',
             ha='center', va='bottom', fontsize=10)
ax2.set_ylim(130, 170)

# ── Plot 3: Top States by Average Price ──
ax3 = axes[1, 0]
states = ['North Carolina', 'California', 'Florida', 'Hawaii']
state_prices = [235.83, 226.55, 200.93, 172.14]
colors3 = ['#e74c3c', '#e67e22', '#f1c40f', '#2ecc71']
bars3 = ax3.barh(states, state_prices, color=colors3, edgecolor='white', linewidth=
ax3.set_title('Top States by Average Price', fontweight='bold', fontsize=12)
ax3.set_xlabel('Average Price ($)')
for bar in bars3:
    ax3.text(bar.get_width() + 2, bar.get_y() + bar.get_height()/2,
             f'${bar.get_width():.0f}',
             ha='left', va='center', fontsize=10)
ax3.set_xlim(0, 280)

# ── Plot 4: Amenity Impact on Price ──
ax4 = axes[1, 1]
amenities = ['Hot Tub', 'Treehouse', 'Luxury', 'Pool', 'Pets']
amenity_diff = [75.77, 42.55, 34.96, -6.58, -7.25]
colors4 = ['#2ecc71' if x > 0 else '#e74c3c' for x in amenity_diff]
bars4 = ax4.barh(amenities, amenity_diff, color=colors4, edgecolor='white', linewid
ax4.set_title('Amenity Impact on Price (vs no amenity)', fontweight='bold', fontsiz
ax4.set_xlabel('Price Difference ($)')
ax4.axvline(x=0, color='black', linewidth=0.8)
for bar, val in zip(bars4, amenity_diff):
    label = f'+${val:.0f}' if val > 0 else f'-${abs(val):.0f}'
    ax4.text(val + (2 if val > 0 else -2), bar.get_y() + bar.get_height()/2,
             label, ha='left' if val > 0 else 'right', va='center', fontsize=10)

# ── Plot 5: Average Price by Property Type ──
ax5 = axes[2, 0]
```

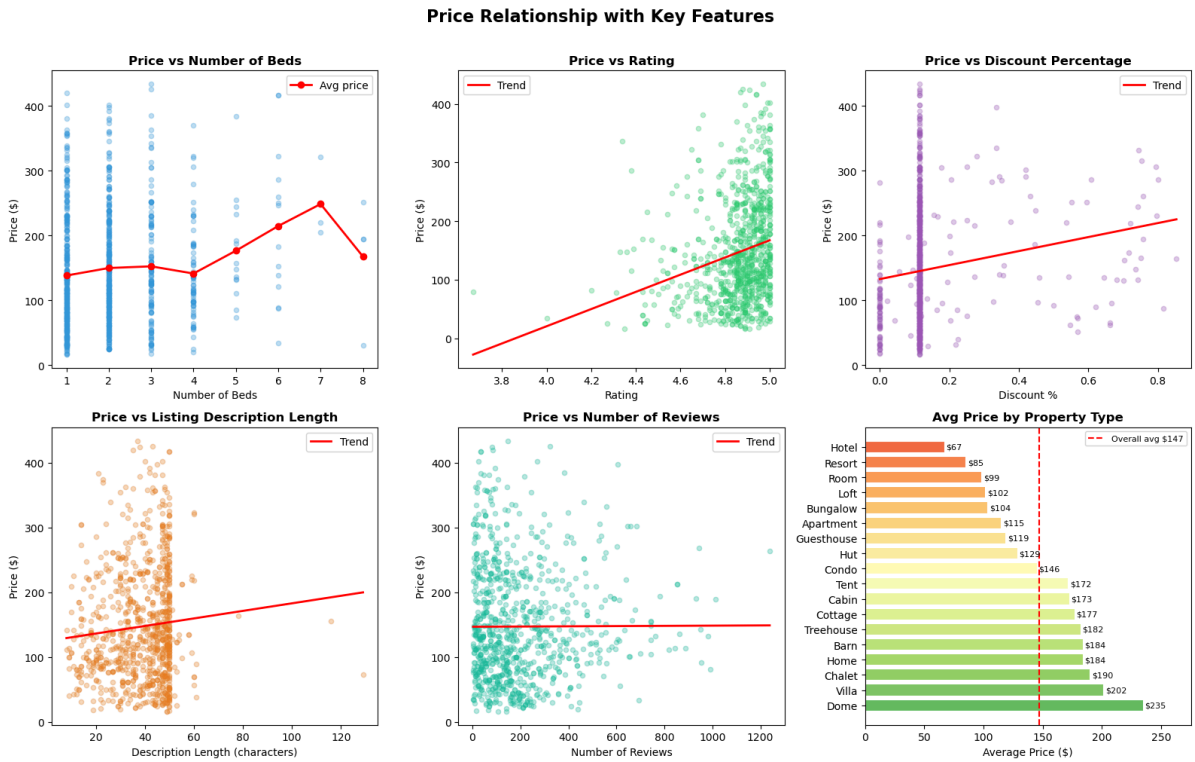
```
prop_types = ['Dome', 'Villa', 'Chalet', 'Home', 'Treehouse',
              'Cottage', 'Cabin', 'Apartment']
prop_prices_static = [235.27, 201.87, 190.33, 184.23, 182.33, 177.42, 172.88, 115.3
colors5 = plt.cm.RdYlGn(np.linspace(0.8, 0.2, len(prop_types)))
bars5 = ax5.bar(prop_types, prop_prices_static, color=colors5, edgecolor='white', l
ax5.set_title('Average Price by Property Type', fontweight='bold', fontsize=12)
ax5.set_ylabel('Average Price ($)')
ax5.set_xticks(range(len(prop_types)))
ax5.set_xticklabels(prop_types, rotation=35, ha='right', fontsize=8)
for bar in bars5:
    ax5.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
             f'${bar.get_height():.0f}',
             ha='center', va='bottom', fontsize=7)
ax5.set_ylim(0, 270)

# ── Plot 6: Benchmark Comparison ──
ax6 = axes[2, 1]
benchmarks = ['BM1\nDefault HP\n(537)', 'BM2\nTuned HP\n(537)']
rmse_values = [71.84, 66.77]
colors6 = ['#e74c3c', '#2ecc71']
bars6 = ax6.bar(benchmarks, rmse_values, color=colors6, edgecolor='white', linewidt
ax6.set_title('Benchmark Comparison — Validation RMSE', fontweight='bold', fontsize
ax6.set_ylabel('Validation RMSE (lower is better)')
for bar, val in zip(bars6, rmse_values):
    ax6.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
             f'{val:.2f}', ha='center', va='bottom', fontsize=11, fontweight='bold'
ax6.set_ylim(60, 80)
ax6.annotate('', xy=(1, 66.77), xytext=(0, 71.84),
             arrowprops=dict(arrowstyle='->', color='black'))
ax6.text(0.5, 68, '↓5.07', ha='center', fontsize=9, color='black')

plt.tight_layout()
plt.savefig("airbnb_insights.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Insights visualization saved as airbnb_insights.png")

# ── Price Relationship with Key Features ──
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Price Relationship with Key Features',
             fontsize=16, fontweight='bold', y=1.01)

# ── Plot 1: Price vs Beds ──
ax1 = axes[0, 0]
beds_avg = df_clean.groupby('beds')['price'].mean().reset_index()
beds_avg = beds_avg[beds_avg['beds'] <= 8]
ax1.scatter(df_clean[df_clean['beds']<=8]['beds'],
            df_clean[df_clean['beds']<=8]['price'],
            alpha=0.3, color='#3498db', s=20)
ax1.plot(beds_avg['beds'], beds_avg['price'],
         color='red', linewidth=2, marker='o', label='Avg price')
ax1.set_title('Price vs Number of Beds', fontweight='bold')
ax1.set_xlabel('Number of Beds')
ax1.set_ylabel('Price ($)')
ax1.legend()

# ── Plot 2: Price vs Rating ──
```

```
ax2 = axes[0, 1]
ax2.scatter(df_clean['rating'], df_clean['price'],
            alpha=0.3, color='#2ecc71', s=20)
z = np.polyfit(df_clean['rating'], df_clean['price'], 1)
p = np.poly1d(z)
x_line = np.linspace(df_clean['rating'].min(), df_clean['rating'].max(), 100)
ax2.plot(x_line, p(x_line), color='red', linewidth=2, label='Trend')
ax2.set_title('Price vs Rating', fontweight='bold')
ax2.set_xlabel('Rating')
ax2.set_ylabel('Price ($)')
ax2.legend()

# ── Plot 3: Price vs Discount ──
ax3 = axes[0, 2]
ax3.scatter(df_clean['discount_pct'], df_clean['price'],
            alpha=0.3, color='#9b59b6', s=20)
z2 = np.polyfit(df_clean['discount_pct'], df_clean['price'], 1)
p2 = np.poly1d(z2)
x_line2 = np.linspace(df_clean['discount_pct'].min(),
                      df_clean['discount_pct'].max(), 100)
ax3.plot(x_line2, p2(x_line2), color='red', linewidth=2, label='Trend')
ax3.set_title('Price vs Discount Percentage', fontweight='bold')
ax3.set_xlabel('Discount %')
ax3.set_ylabel('Price ($)')
ax3.legend()

# ── Plot 4: Price vs Detail Length ──
ax4 = axes[1, 0]
ax4.scatter(df_clean['detail_length'], df_clean['price'],
            alpha=0.3, color='#e67e22', s=20)
z3 = np.polyfit(df_clean['detail_length'], df_clean['price'], 1)
p3 = np.poly1d(z3)
x_line3 = np.linspace(df_clean['detail_length'].min(),
                      df_clean['detail_length'].max(), 100)
ax4.plot(x_line3, p3(x_line3), color='red', linewidth=2, label='Trend')
ax4.set_title('Price vs Listing Description Length', fontweight='bold')
ax4.set_xlabel('Description Length (characters)')
ax4.set_ylabel('Price ($)')
ax4.legend()

# ── Plot 5: Price vs Num Reviews ──
ax5 = axes[1, 1]
ax5.scatter(df_clean['num_reviews'], df_clean['price'],
            alpha=0.3, color='#1abc9c', s=20)
z4 = np.polyfit(df_clean['num_reviews'], df_clean['price'], 1)
p4 = np.poly1d(z4)
x_line4 = np.linspace(df_clean['num_reviews'].min(),
                      df_clean['num_reviews'].max(), 100)
ax5.plot(x_line4, p4(x_line4), color='red', linewidth=2, label='Trend')
ax5.set_title('Price vs Number of Reviews', fontweight='bold')
ax5.set_xlabel('Number of Reviews')
ax5.set_ylabel('Price ($)')
ax5.legend()

# ── Plot 6: Avg Price by Property Type (dynamic) ──
ax6 = axes[1, 2]
```

```
prop_cols = [c for c in df_clean.columns if 'property_type_' in c]
prop_prices = {}
for col in prop_cols:
    subset = df_clean[df_clean[col] == True]['price']
    if len(subset) >= 5:
        prop_prices[col.replace('property_type_', '')] = subset.mean()
prop_prices = dict(sorted(prop_prices.items(), key=lambda x: x[1], reverse=True))
colors_prop = plt.cm.RdYlGn(np.linspace(0.8, 0.2, len(prop_prices)))
bars = ax6.barh(list(prop_prices.keys()), list(prop_prices.values()),
                color=colors_prop, edgecolor='white', linewidth=1.5)
ax6.set_title('Avg Price by Property Type', fontweight='bold')
ax6.set_xlabel('Average Price ($)')
ax6.axvline(x=df_clean['price'].mean(), color='red', linestyle='--', linewidth=1.5,
            label=f'Overall avg ${df_clean["price"].mean():.0f}')
ax6.legend(fontsize=8)
for bar, val in zip(bars, prop_prices.values()):
    ax6.text(val + 1, bar.get_y() + bar.get_height()/2,
             f'${val:.0f}', ha='left', va='center', fontsize=8)
ax6.set_xlim(0, max(prop_prices.values()) + 40)

plt.tight_layout()
plt.show()
```



✅ Insights visualization saved as airbnb_insights.png



Key Insights from Data Analysis:

1. **Season:** I found that Spring listings average 187 dollars — making it the most expensive season to book. Interestingly, Summer had the most listings (583) but the lowest average price at 129 dollars. This surprised me because I expected Summer to be the most expensive season, but high supply seems to drive prices down.

2. **Quarter:** I noticed that Q3 (Jul-Sep) is the most expensive quarter at 154.90 dollars, followed by Q2 at 145.33 dollars. This aligns with typical travel patterns where late summer bookings are slightly more expensive than early summer.

3. **Location:** From my analysis, North Carolina (235 dollars) and California (227 dollars) command the highest prices. This told me that location is the strongest single driver of Airbnb pricing in my dataset.

4. **Amenities:** I was surprised to find that a hot tub adds the most value to a listing at plus 75.77 dollars, more than a luxury label at plus 34.96 dollars. I also noticed that having a pool or being pet-friendly actually showed a slight negative correlation with price, which I did not expect.

5. **Property Type:** My data showed that Dome (235 dollars) and Villa (202 dollars) are the most premium property types, while Hotels (67 dollars) and Rooms (98 dollars) are the most affordable — guests clearly pay more for unique experiences.

6. **Benchmarks:** The most important thing I learned from this project is that feature engineering, which improved RMSE by 5.07 points from 71.84 to 66.77, had a bigger

impact than hyperparameter tuning alone. This showed me that investing time in meaningful data preparation is more valuable than just tuning model parameters.

In [ ]:

```

```